Journal of Biomedical Informatics 135 (2022) 104219

Contents lists available at ScienceDirect

Journal of Biomedical Informatics

journal homepage: www.elsevier.com/locate/yjbin

Original Research

Detecting anomalous sequences in electronic health records using higher-order tensor networks☆

Haoran Niu [a], Olufemi A. Omitaomu [a,b,*], Michael A. Langston [a], Mohammad Olama [b], Ozgur Ozmen [b], Hilda B. Klasky [b], Angela Laurio [c], Brian Sauer [c], Merry Ward [c], Jonathan Nebeker [c]

[a] University of Tennessee, Knoxville, TN, 37996, United States
[b] Oak Ridge National Laboratory, Oak Ridge, TN, 37831, United States
[c] Department of Veterans Affairs, Washington DC, 20420, United States

ARTICLE INFO

Keywords:
Event sequence
Anomaly detection
Higher-order tensor network
Electronic health records

ABSTRACT

Detecting anomalous sequences is an integral part of building and protecting modern large-scale health information technology (HIT) systems. These HIT systems generate a large volume of records of patients' state and significant events, which provide a valuable resource to help improve clinical decisions, patient care processes, and other issues. However, detecting anomalous sequences in electronic health records (EHR) remains a challenge in healthcare applications for several reasons, including imbalances in the data, complexity of relationships between events in the sequence, and the curse of dimensionality. Conventional anomaly detection methods use the finite sequence of events to discriminate sequences. They fail to incorporate salient event details under variable higher-order dependencies (e.g., duration between events) that can provide better discrimination of sequences in their models. To address this problem, we propose event sequence and subsequence anomaly detection algorithms that (1) use network-based representations of interactions in the data, (2) account for variable higher-order dependencies in the data, and (3) incorporate events duration for adequate discrimination of the data. The proposed approach identifies anomalies by monitoring the change in the graph after the test sequence is removed from the network. The change is quantified using graph distance metrics so that dramatic changes in the network can be attributed to the removed sequence. Furthermore, the proposed subsequence algorithm recommends plausible paths and salient information for the detected anomalous subsequences. Our results show that the proposed event sequence anomaly detection algorithm outperforms the baseline methods for both synthetic data and real-world EHR data.

1. Introduction

Sequence data occur naturally in many real-world applications; thus, detecting anomalies in sequence data becomes an important task. Anomaly detection methods identify anomalies in the sequence by characterizing the underlying patterns in the data. A *sequence* is defined as an ordered series of events [1]. Many high-impact applications have sequence anomaly detection problems, including bioinformatics [2], healthcare [3,4], cybersecurity [5,6], and financial fraud detection [7], among others [8]. In the age of big data, efficiently identifying anomalies or implausible patterns in sequence data has become more critical than ever.

Anomaly detection techniques for healthcare sequential data can support human decisions and mitigate any potential hazards by extracting irregular patterns and presenting them in a useful form. Most health information technology (HIT) systems do not generate electronic health record (EHR) logs, which are a valuable resource for understanding and monitoring the state of the healthcare process. Thus, sequence data,

☆ This work is sponsored by the US Department of Veterans Affairs. This manuscript has been authored by UT-Battelle, LLC, under contract DE-AC05-00OR22725 with the US Department of Energy (DOE). The US government retains and the publisher, by accepting the article for publication, acknowledges that the US government retains a nonexclusive, paid-up, irrevocable, worldwide license to publish or reproduce the published form of this manuscript, or allow others to do so, for US government purposes. DOE will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan (http://energy.gov/downloads/doe-public-access-plan).

* Corresponding author at: Oak Ridge National Laboratory, Oak Ridge, TN, 37831, United States.
E-mail address: omitaomuoa@ornl.gov (O.A. Omitaomu).

https://doi.org/10.1016/j.jbi.2022.104219
Received 18 August 2022; Accepted 3 October 2022
Available online 13 October 2022
1532-0464/© 2022 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

obtained using reverse engineering approaches, records patient care status and significant events to help capture healthcare processes and perform further analysis. Therefore, online monitoring and anomaly detection of EHR sequential data is of great interest to improve the clinical decisions and patient care processes. The sequence of events in an HIT system reveals the footprint of the system. When anomalous behavior occurs, the implicit patterns of the sequence deviate from the normal data.

Plenty of research has been dedicated to sequence-based anomaly detection. Point-based techniques that compute the similarity between sequences and present an anomaly score (e.g., kth nearest neighbor [kNN] [9], k-means [4], and clustering methods [10–12]) are well studied in the time-series anomaly detection domain. Feature-embedding methods (e.g., word2vec [13], sequence graph transform [SGT] [14], and recurrent neural networks [15,16]) are required to provide machine-interpretable representations for sequences before adapting point-based techniques in event-sequence anomaly detection. Markov-based methods, in which all transitions between events are analyzed, are widely used for sequence anomaly detection. Fixed Markovian methods (e.g., extended finite state automata [17], suffix trees [18]) use a fixed history of sequences to calculate the conditional probability of an event. However, these methods lack the flexibility to provide a reliable estimate of transition probability and require a huge amount of space to store all transitions. Variable Markov-based approaches are proposed to address these challenges by storing variable length histories of sequences [19–21]. Other popular methods include the hidden Markov model–based approaches [22–24] and neural network–based methods [25–27].

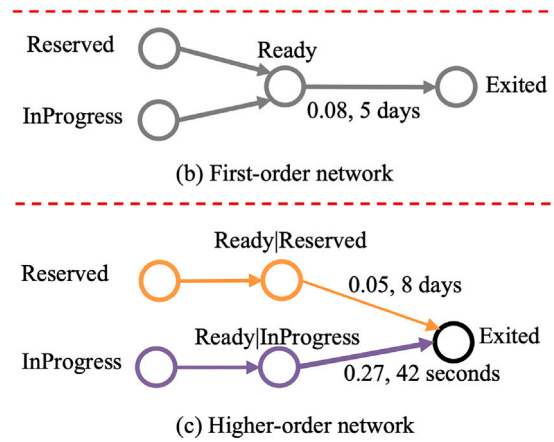
However, these existing methods ignore additional – and likely valuable – information in high-dimensional sequence data. Examples of such additional information are the duration between prescription dosages in medical records, two events in a cyber network, and the location of events in credit card fraud detection. The additional information is important because it provides a description and detail for a given event, which can further characterize the individual event and help identify anomalies in the data. In this paper, a higher-order tensor network (HOTN) is proposed to efficiently build accurate network representations by capturing both the traditional transition probability and the salient transition information under high-order dependencies. HOTN can pick up the potential higher-order information that was missed in the first-order network representation by considering the current event for the next movement along with the previous several events.

Motivating Example: Fig. 1 shows an example from our real-world dataset of a radiology test order from an ischemic heart disease cohort. The table in Fig. 1a shows that the transition probability from `Ready` to `Exited` is 0.08, and the average transition duration is 5 days. However, more accurate information could be leveraged by considering the previous events (high-order dependencies): if the previous event of `Ready` is `Reserved`, then the transition probability is 0.05 with 8 days as the average duration to `Exited`; if the previous event of `Ready` is `InProgress`, then the transition probability is 0.27 with 42 s as the average duration to `Exited`. By splitting the `Ready` node into two higher-order nodes, the network representation is more accurate and more insightful.

This additional information can also be used to design triggers (e.g., abnormal duration between events, potential drug overdoses, drug interactions in patients) for monitored systems which still remain a challenge. To integrate this additional information into a knowledge discovery process, the authors propose the Anomaly Detection algorithm based on Higher-Order Tensor Networks (AD-HOTN), an unsupervised method that uses a HOTN representation to fully characterize data and detect anomalous sequences in high-dimensional event sequence data. The authors also propose the Contiguous Subsequence Anomaly Detection algorithm based on Higher-Order Tensor Networks (CSAD-HOTN) to locate the abnormal subsequences within the detected anomalous sequences. The main contributions of this work are summarized as follows:

ID	EHR Data	Date Time
0	Reserved	2017-5-12 10:12:30
0	Ready	2017-5-18 15:05:03
0	Exited	2017-5-22 11:12:30
1	InProgress	2017-5-12 10:12:30
…	…	…

(a) Raw EHR data



(b) First-order network

(c) Higher-order network

Fig. 1. Representing a sequence in an HOTN.

- A network representation method, HOTN, is introduced to fully capture the potential information of high-dimensional event sequence data. We demonstrate that HOTN is efficient in scaling to higher orders because the order grows only when necessary.
- A graph-based novel framework, AD-HOTN, is proposed to explicitly identify anomalies in data sequences. The method can detect abnormal pathways and anomalies in salient information. This framework is also implemented on the Apache Spark platform to increase its scalability to big sequence data applications.
- A subsequence anomaly detection algorithm, CSAD-HOTN, is proposed to accurately locate the abnormal segments within the anomalous sequences.
- The proposed methods are demonstrated to outperform the state-of-art baseline methods by using synthetic sequence data and real-world EHRs.

The remainder of this paper is organized as follows: Section 2 introduces some notations and concepts used throughout the paper, defines three related problems in sequence-anomaly detection, and states two hypotheses that are the cornerstones of the algorithms. Section 3 presents the proposed algorithms and describes the implementation of the algorithms via Apache Spark. Section 4 describes the evaluation of the proposed algorithms against the state-of-the-art baseline methods using synthetic and real-world data. Section 5 summarizes some related studies. Section 6 concludes the paper with a brief summary.

2. Preliminaries

This section presents notations and basic concepts used in this paper. It also clarifies three problem formulations for sequence-based anomaly detection, contiguous subsequence-based anomaly detection, and normal subsequences recommendation. Then, it presents the two hypotheses that are the cornerstones of the algorithms.

Definition 1. A *sequence* refers to a discrete sequence in this paper and is defined as a finite sequence of events [1]. Additional information (e.g., timestamp) usually accompanies each event. Sets of sequences will be denoted using bold uppercase letters (e.g., **S**, **T**). Individual sequences will be denoted using bold lowercase letters (e.g., **s**, **t**). The length of a sequence, **s**, will be denoted as l_s.

Definition 2. An *event* is defined as a symbol, which is the smallest meaningful element in a sequence. An event within a sequence, **s**, will be denoted as e [1].

Definition 3. A *subsequence* is defined as a smaller sequence that can be derived from the original sequence by deleting some events/symbols without changing the order of the remaining symbols [28]. In this paper, a subsequence refers to a *contiguous subsequence* of a sequence. Formally, a subsequence of a sequence, **s**, is denoted as $s_{i:j}$, which starts from the ith location and ends at the jth location of the sequence, **s**.

Definition 4. A *transition*, which is the smallest subsequence in a sequence, is defined as a movement between two contiguous events within a sequence.

Definition 5. A *tensor* can be represented as a multidimensional array and is a higher-way generalization of a vector and a matrix [29]. In this paper, for a network consisting of N nodes, transitions are represented as 4D tensors, $\mathscr{T} \in \mathbb{R}^{N\times N\times P\times D}$ (source, target, transition probability, and transition duration). This is called a *tensor network*, which will be denoted as $\mathscr{N}$.

Problem Definition 1 (*Sequence-based Anomaly Detection*). *Given a set of n sequences, $\boldsymbol{S} = \{\boldsymbol{s}_1,\boldsymbol{s}_2, \ldots,\boldsymbol{s}_n\}$, find anomalous sequences in **S** with respect to the rest of **S**.*

Problem Definition 2 (*Subsequence-based Anomaly Detection*). *Detect contiguous subsequences in a detected abnormal sequence, **s**, that are anomalous with respect to the rest of **s**.*

Problem Definition 3 (*Normal Subsequences Recommended for Detecting Abnormal Subsequences*). *Based on the historical normal data patterns in a given set of sequences, **S**, detected abnormal subsequences, $s_{i:j}$, are replaced by recommend subsequences, $\overline{s}_{i:j}$.*

Hypothesis 1. Given a sequence set, **S**, which contains a large amount of sequences, anomalous or implausible sequences should be sparse.

Hypothesis 2. Higher-order dependencies, which assume that the next event depends on the current event and several previous events, are highly permeated in many applications.

3. Methods

This section details the proposed algorithms. First, the HOTN representation is presented for high-dimensional sequence data. Second, the AD-HOTN event sequence anomaly detection algorithm, in which the algorithm can detect anomalies based on abnormal transitions and implausible transition durations, is presented. Third, to identify an abnormal contiguous subsequence from the detected anomalies, the CSAD-HOTN is presented. The combination of approaches will flag anomalous sequences with implausible pathways or transition durations, flag abnormal subsequences within the flagged sequences, and propose plausible transitions and durations for anomalies.

3.1. Representing event sequence data in higher-order tensor networks

The higher-order network (HON) representation [30] was proposed to ensure accurate representation of the underlying phenomena in complex systems. HON representations eliminate the potential information loss introduced by a conventional Markov network representation [30]. However, the HON representation only contains transition probabilities or frequencies without accounting for transition duration or other pertinent information in the data. To address this limitation, the authors present the HOTN representation for multidimensional event sequence data. The approach can preserve more information in the data by using transition probabilities and average transition durations as an edge weight. This method also scales to more variables.

Generally, a network is a graph represented as $G = (V, E, \psi)$, where V represents vertices or nodes, and E represents edges or links (i.e., connections between pairs of nodes). If the event in a network at time t is denoted as $e_t(n_t \in V)$, then the transition probability of movement from node n_t to node n_{t+1} is proportional to the count between the two nodes, $C(n_t \to n_{t+1})$, which is given as

$$P(e_{t+1} = n_{t+1}|e_t = n_t) = \frac{C(n_t \to n_{t+1})}{\sum_j C(n_t \to n_j)}, \tag{1}$$

where $n_j \in V$. The probability of a movement between two nodes is part of the edge weight of the network. An edge weight of a transition can also preserve other information (e.g., average transition duration). If the timestamp of node n_t is ts_t, and the timestamp of node n_{t+1} is ts_{t+1}, then the transition duration $D_s(n_t \to n_{t+1})$, which is calculated as $ts_{t+1} - ts_t$, represents the time duration of movement from nodes n_t to n_{t+1} in sequence **s**. In a sequence set, **S**, the average duration (AvgD) between nodes n_t and n_{t+1}, where the event in a network at time t is denoted as $e_t(n_t \in V)$, is given as

$$AvgD(e_{t+1} = n_{t+1}|e_t = n_t) = \frac{\sum_{s\in \mathbf{S}} D_s(n_t \to n_{t+1})}{C(n_t \to n_{t+1})}. \tag{2}$$

For the tensor network representation, a vector is assumed to contain the transition probability, Eq. (1), and average duration, Eq. (2), as the edge weight. The vector can account for more variables.

The conventional network representation assumes that the next event depends only on the current event, Eq. (2), thereby ignoring that previous events can also influence the next event [30]. Consequently, the Markovian assumption has been well studied in network representation [17,18,31], in which each node can also represent a sequence of nodes. A node that contains its contiguous historical nodes is defined as a higher-order node, and the node preserves higher-order dependencies. A kth-order node is denoted as $n_t|n_{t-1}...n_{t-k-1}$, where the previous movements of node n_t are $n_{t-k-1} \to \cdots \to n_{t-1}$. The more previous events contained in a node, the higher its order dependencies.

Hence, the probability and average duration of moving to the next node, n_{t+1}, from a higher-order node, $n_t|n_{t-1}...n_{t-k}$, are given as

$$\begin{aligned} &P(e_{t+1} = n_{t+1}|e_t = (n_t|n_{t-1}...n_{t-k})) \\ &\quad = \frac{C(n_t|n_{t-1}...n_{t-k} \to n_{t+1})}{\sum_\ell C(n_t|n_{t-1}...n_{t-k} \to n_\ell)}, \end{aligned} \tag{3}$$

where $n_\ell \in V$ and $k < t$,

$$\begin{aligned} &AvgD(e_{t+1} = n_{t+1}|e_t = (n_t|n_{t-1}...n_{t-k})) \\ &\quad = \frac{\sum_{s\in \mathbf{S}} D_s((n_t|n_{t-1}...n_{t-k}) \to n_{t+1})}{C((n_t|n_{t-1}...n_{t-k}) \to n_{t+1})}. \end{aligned} \tag{4}$$

Xu et al. [20,30] proposed BuildHON+ for an HON representation of sequence data. A key step in BuildHON+ is to determine whether increasing a node's order from a lower-order transition to a higher-order transition will produce significant dependencies. Kullback–Leibler divergence between current-order and higher-order nodes is used to weight the significance. The change is significant if it satisfies the following divergence criterion [20]:

$$\mathscr{D}_{KL}(P_{cur}, P_{ExtNode}) > \frac{Order_{ExtNode}}{\log_2(1 + Support_{ExtNode})}, \tag{5}$$

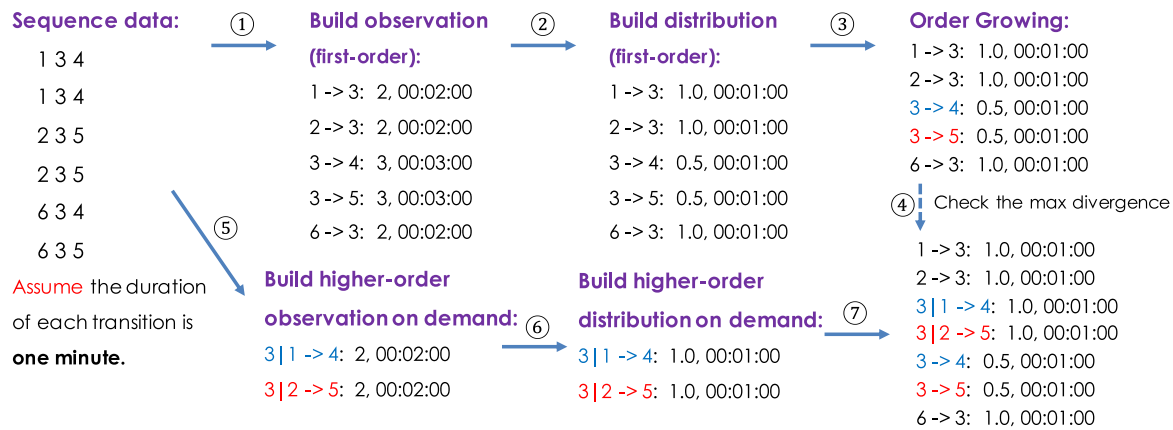


Fig. 2. Example of BuildHOTN with transition probability and duration as the edge weight. The process order is labeled with circled numbers.

where P_{cur} is a set of distributions of the current node, $P_{ExtNode}$ is a set of distributions of the extended node, and $Support_{ExtNode}$ is the number of observations of the extended node.

Adapting the idea of building a network from BuildHON+, we designed BuildHOTN for HOTN representations for multidimensional event sequence data. BuildHOTN uses a tensor network representation that can preserve more information, which leads to higher accuracy. Specifically, frequency and transition probability were used as the edge weight in the network representation, but more values (e.g., transition duration) were also included in the edge weight.

Fig. 2 illustrates the BuildHOTN process, in which there are six events in six sequence data, and each sequence has three events. The duration of each transition is assumed to be 1 min. BuildHOTN first builds all first-order observations, which contain counts and the total duration of the transition (step ①). Then, all first-order distributions are built, and each edge has the transition probability and average duration (steps ②→③). Because the Kullback–Leibler divergence is less than the threshold for paths $1 \rightarrow 3$, $2 \rightarrow 3$, and $6 \rightarrow 3$, which suggests that they have no higher-order dependencies, those paths will not be extended. Only paths $3 \rightarrow 4$ and $3 \rightarrow 5$ will be extended (step ④). Because the second-order observations and distributions of the two paths are not yet known, BuildHOTN selectively builds second-order distributions that significantly change the raw sequence data (steps ⑤→⑦). At this point, BuildHOTN stops building because there is no dependencies higher than second order. Otherwise, BuildHOTN will continue building (i.e., growing) until it reaches the maximum order.

3.2. Sequence anomaly detection based on higher-order tensor networks

The AD-HOTN algorithm presented in Alg. 1 uses a HOTN representation and graph-distance metrics. Fig. 3 also presents a diagram of the AD-HOTN method when given a sequence set, $\mathbf{S} = \{\mathbf{s}_1, \mathbf{s}_2, \ldots, \mathbf{s}_n\}$, which contains n sequences along with their event timestamps. First, the sequential data, $\mathbf{S}$, are represented as a tensor network, $\mathcal{N}_s$ (line 3, Alg. 1). Then, by removing a sequence $\mathbf{s}_i$ ($i \in (0, n]$) from data $\mathbf{S}$ each time, HOTN $\mathcal{N}_i$ is built for the sequence set $\mathbf{S}_i$ ($\mathbf{S} \backslash \{\mathbf{s}_i\}$) (lines 4–5, Alg. 1). By iterating the n sequences in $\mathbf{S}$, n HOTNs are built, $N_s = \{\mathcal{N}_1, \mathcal{N}_2, \ldots, \mathcal{N}_n\}$ (line 6, Alg. 1). The authors removed sequence $\mathbf{s}_i$ from sequence set $\mathbf{S}$ because the network change will be revealed by comparing network $\mathcal{N}_s$ with network $\mathcal{N}_i$. If a normal sequence is removed, then the network will change less than when an outlier is removed. Because outliers always contain unique data patterns, eliminating those unique patterns from the network will remove some nodes or edges and cause large network changes.

The network changes are then quantified by calculating the network differences using network distance metrics, $\mathcal{ND}$ [32]. The network metrics reveal the dissimilarity between two networks. The larger the network distance, the more dissimilar the networks are, and the more likely that the removed sequence, $\mathbf{s}_i$, is an anomaly. This work used the weight distance metric [32]; thus, the network distance between network G and network H is given as

$$\mathcal{ND}(G, H) = |E_G \cup E_H|^{-1} \sum_{u,v \in V} \frac{|w_E^G(u,v) - w_E^H(u,v)|}{\max(w_E^G(u,v), w_E^H(u,v))}, \tag{6}$$

where $w_E^G(u,v)$ is the edge weight between nodes u and v for network G, and $|E_G \cup E_H|$ is the value of the union of edges in networks G and H.

Thus, the network distance between $\mathcal{N}_s$ and $\mathcal{N}_i$ will be calculated as $ND_i = \mathcal{ND}(\mathcal{N}_i, \mathcal{N}_s)$ (line 7, Alg. 1). ND_i contains the network change transition probability and average duration (ND_i^P, ND_i^{AD}) because the edge weight contains the transition probability, P, Eq. (3), and average transition duration, $AvgD$, Eq. (4). By iterating all networks in N_s, the network distance set, $ND_S = \{ND_1, ND_2, \ldots, ND_n\}$, is obtained (line 7, Alg. 1).

Detecting anomalies in sequences then becomes anomaly detection in point data, ND_S. Gaussian anomaly detection was applied to identify the anomalous sequences with transitions and/or duration. The threshold is calculated as $threshold = \text{Mean}(ND_S) + 2 * \text{STD}(ND_S)$, where STD refers to standard deviation (line 9, Alg. 1). If the network distance, ND_i, of the transition probability is larger than its threshold, then it will be identified as an anomaly with abnormal transitions (line 10, Alg. 1). Similarly, if the network distance, ND_i, of the transition duration is larger than its threshold, then it will be identified as an anomaly with abnormal duration (line 11, Alg. 1).

Algorithm 1 AD-HOTN.

```
1: procedure AD_HOTN(S)
2:     ND_s ← [ ]                                  ▷ a list of pairs of network distances
3:     N_s ←BuildHOTN(S)                           ▷ build HOTN for S
4:     for i = 1 → |S| do                          ▷ |S| is the number of sequences
5:         S_i ← S.pop(s_i)                        ▷ remove sequence s_i from S
6:         N_i ← BuildHOTN(S_i)                    ▷ build HOTN for S_i
7:         ND_i ← ND(N_i, N_s)                     ▷ calculate network
                                                   distance between N_i and N_s
8:         append(ND_i) to ND_s
9:     threshold ← mean(ND_S) + 2 * std(ND_S)
10:    pred_p ← [1 if nd[0] > threshold[0] else 0 for nd in ND_s]   ▷ anomalies with
       abnormal transition
11:    pred_d ← [1 if nd[1] > threshold[1] else 0 for nd in ND_s]   ▷ anomalies with
       abnormal duration
12:    pred ← zip(pred_p, pred_d)                  ▷ zip two predictions
13:    return pred                                 ▷ return prediction
```

3.3. Contiguous subsequence anomaly detection based on higher-order tensor networks

This section introduces the CSAD-HOTN approach for identifying anomalous subsequences within an anomalous sequence. The authors

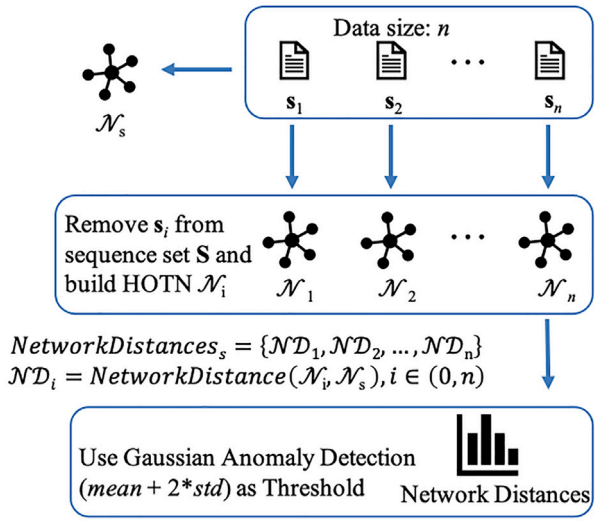


Fig. 3. Diagram of anomaly detection based on HOTN.

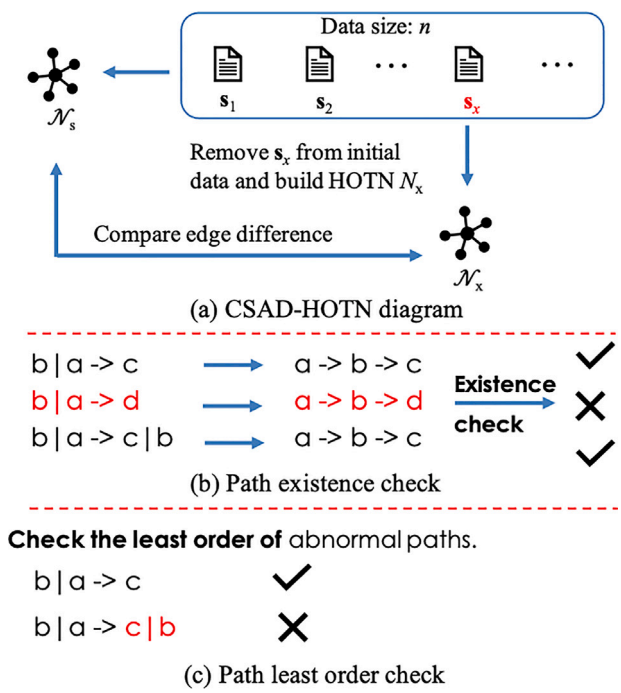


Fig. 4. Diagram of CSAD-HON.

aim to (1) identify anomalous transitions and durations within the outliers detected by AD-HOTN and (2) predict the most plausible transition and duration for each anomalous transition and duration. Unfortunately, the existing subsequence detection algorithm (e.g., CSAD [21]) is not suited for this purpose because the conventional HON does not preserve enough information. The HOTN representation enables the opportunity for duration detection by carrying transition duration information in the network. CSAD-HOTN can also scale to higher dimensional sequence data beyond just transition probability and duration.

Algorithm 2 explains CSAD-HOTN in detail. To detect abnormal transitions and durations for an anomaly, $\mathbf{s}_x$, the authors take advantage of the $\mathcal{N}_x$ and $\mathcal{N}_s$ generated in AD-HOTN; this step is illustrated in Fig. 4a. The authors compare the edges in $\mathcal{N}_x$ and $\mathcal{N}_s$ and identify those with large differences between the two networks. Specifically, if the change in transition probability of an edge is larger than a threshold, then the edge is identified as an abnormal transition; if the change of transition duration is greater than a threshold, then the edge has an abnormal transition duration. The edge difference is quantified using the edge distance metric between edges of the network $\mathcal{N}_x$ and $\mathcal{N}_s$ (lines 20–26, Alg. 2). The edge distance between the edges of networks G and H is given as

$$\text{EdgeDistance}(G, H) = \frac{|w_E^G - w_E^H|}{\max(w_E^G, w_E^H)}, \tag{7}$$

where w_E is the weight for edge E (line 21, Alg. 2). If an edge is not in a network, then the edge weight will be (0, 0) (lines 23–24, Alg. 2). By using Gaussian anomaly detection, abnormal transitions and durations are identified (lines 30–31, Alg. 2), where the threshold is calculated as $threshold = \text{Mean} + 2 * \text{STD}$ (standard deviation).

After the abnormal transitions and durations are identified in $\mathbf{s}_x$, the existence of the abnormal paths in the sequence are verified (line 8, Alg. 2). Then, only the least-order edges with the same source node are kept (line 9, Alg. 2). For example, in Alg. 2b, if edges $(b|a \to c|)$, $(b|a \to d|)$, and $(b|a \to c|b)$ are detected, then edge $(b|a \to d)$ will be discarded because the subsequence does not exist in sequence $\mathbf{s}_x$. In addition, edge $(b|a \to c|b)$ will be discarded because node $c|b$ is a higher-order node of $c|$ (i.e., both are from the same root node, c), and the least-order node is kept to retain more information and reduce redundancy. After detecting the abnormal transitions or durations, CSAD-HOTN also recommends appropriate transitions (line 10, Alg. 2) and durations (line 12, Alg. 2) according to the edge weight in the network. By searching the edges with the same source node in the network, $\mathcal{N}_x$, CSAD-HOTN recommends plausible transitions according to the transition probability of edge weights — a larger transition probability of the edge weight means higher probability for the recommended transition. Similarly, an appropriate duration is also recommended by searching the path in the network, $\mathcal{N}_x$.

3.4. Spark-based implementation

To achieve algorithm scalability on big sequence data, the authors implemented the algorithms on the Apache Spark platform [33], which is an open-source software framework that supports big data applications on distributed machines. In this study, Apache Spark was used to process network representations for big sequence data while speeding up the anomaly detection process.

The size of a raw sequence data is given as L when building HOTNs, all first-order dependencies take $\Theta(2L)$ space, second-order dependencies take $\Theta(3L)$ space, and so on. Assuming that the number of kth-order dependencies of each sequence is R_k $(1 \le R_k < L)$, K is the maximum order, and N is the total number of sequences, then the space complexity to build an HOTN for N sequence is given as $\mathcal{O}\left(N \cdot \sum_{k=1}^{K}(k+1)R_k\right) < \mathcal{O}\left(N \cdot \sum_{k=1}^{K}(k+1)L\right) = \mathcal{O}(NK^2L)$. Because the number of networks to build is N in AD-HOTN, the time complexity is $\mathcal{O}(N^2K^2L)$. If there are m executors when Spark-based AD-HOTN is deployed, then the time complexity of Spark-based AD-HOTN is given as $C_{\text{ADSpark}} = \mathcal{O}(N^2K^2L/m)$. Thus, the overall time complexity of Spark-based AD-HOTN will be significantly smaller when significantly more executors, m, are deployed. The side-by-side performance comparison between prior work [3] and Spark-based AD-HOTN is presented in Section 5.

4. Evaluations and discussions

This section discusses the experiments on two datasets. The first was a synthetic dataset used to compare the performance of the authors' proposed methods against the baseline methods. The second was a real-world dataset from a healthcare application that consisted of clinical consultation events in the US Department of Veterans Affairs (VA) healthcare facilities.

Algorithm 2 CSAD-HOTN.

Input: s_x: detected anomalous sequence; *pred*: prediction of s_x; $\mathcal{N}_s$: HOTN of S; $\mathcal{N}_x$: HOTN of S after removing s_x.
Output: AT: subsequences of s_x that have abnormal transition; RT: Recommend transitions. AD: subsequences of s_x that have abnormal duration; RD: Recommend duration.

```
procedure CSAD(N_s, N_x)
    Edges_s ← ExtractEdges(N_s)
    Edges_x ← ExtractEdges(N_x)
    AT, AD ← AnomalyDetect(Edges_s, Edges_x)
    if (pred[0] and pred[1]) then
        AD ← remove item from AD that exists in AT
    if pred[0] then                                   ▷ if s_x contains abnormal transition
        AT ← CheckPathExistence(AT)
        AT ← KeepLeastOrderPath(AT)
        RT ← Recommend(AT, Edges_x, 'T')
    if pred[1] then                                   ▷ if s_x contains abnormal duration
        RD ← Recommend(AD, Edges_x, 'D')
    return AT, RT, AD, RD
function ExtractEdges(N)
    for path in N do
        source, target, tp, avgd ← ExtractPath(path)
                                                      ▷ tp: transition probability; avgd: average duration
        Edges[(source, target)] ← (tp, avgd)
    return Edges
function AnomalyDetect(G, H)
    Distance ← [ ]                                    ▷ a list of edges distance info
    AllEdges ← G ∪ H                                  ▷ a list of all edges
    for edge in AllEdges do
        weight_G ← G[edge] if edge in G else (0, 0)
        weight_H ← H[edge] if edge in H else (0, 0)
        distance ← EdgeDistance(weight_G, weight_H)
        append (distance, edge, weight_G) to Distance
    sort Distance by value Distance[0] descending
    d ← Distance[:, 0]                                ▷ a list of all edge distance
    threshold ← mean(d) + 2 * std(d)
    AT ← [[D[1], D[2]] for D in Distance if (D[0][0] > threshold[0])   ▷ identify abnormal transition
    AD ← [[D[1], D[2]] for D in Distance if (D[0][1] > threshold[1])]  ▷ identify abnormal duration
    return AT, AD
function Recommend(Paths, Edges, mode)
    R ← [ ]                                           ▷ a list of recommend paths
    for path in Paths do
        source, target ← ExtractPath(path)
        if mode =='T' then                            ▷ transition mode
            Targets, weights ← Edges(source)
            R.append (source, Targets, weights)
        if mode =='D' then                            ▷ duration mode
            weights ← Edges(path)
            R.append (source, target, weights)
    return R
```

4.1. Baseline methods

For comparison, two point-based predictive methods and three Markov-based methods were selected as the baseline methods (listed below). These methods represent state-of-the-art, and their performance and effectiveness have been demonstrated [3,9,14,21,30,34–36].

- The **SGT** [14] sequence embedding method extracts the short-term and long-term sequence features and embeds them in a finite-dimensional feature space. The SGT was leveraged as an embedding method before adapting the two point–based predictive models.
- The **kNN-based detector** [9] uses an unsupervised approach, even though kNN is a supervised machine learning algorithm. The anomaly score of each sequence equals the dissimilarity to its kNN. After setting a threshold value, outliers will be identified. This method was used as one of the baselines because it is an efficient and effective point-based anomaly detection method.
- The **k-means-based detector** [34] collects points into a fixed number of clusters, and the distance to its closet centroid is calculated for each data point. For each cluster, the data outside the threshold will be counted as an outlier.
- **N-gram** [37,38] is widely studied by the natural language processing community. N-gram takes a fixed historical length of an event into consideration by calculating the probabilities. The idea is that a particular event in a sequence is only influenced by its recent predecessors rather than a single previous event or the entire history. To apply the N-gram model in our setting, we use N as the fixed history order.
- **PST** [19], or a probabilistic suffix tree, is a compact tree representation of a variable Markov chain that uses classical suffix trees as its index structure. Variable Markov techniques advance over the unreliable conditional probability of Markov techniques by allowing symbols to be conditioned on a variable-length history. The PST-based technique computes the likelihood for the test sequence, **s**, with respect to the PST. Given a sequence, $\mathbf{s} = [s_1, \ldots, s_n]$, and a PST, T, the odds of **s** with respect to T are denoted by SIM_O and defined as

$$SIM_O(\mathbf{s}, T) = \frac{\prod_{i=1}^{l_s} P^T(s_i | s_1 \ldots s_{i-1})}{\prod_{i=1}^{l_s} P^T(s_i)}, \tag{8}$$

  where $P^T(s_i)$ is the empirical probability of event s_i in **s**, and l_s is the number of events in sequence **s**. Thus, SIM_O represents the likelihood that **s** belongs to T (as opposed to a random sequence). If the value of $SIM_O(\mathbf{s}, T)$ is less than 1, then it indicates that **s** is more likely to be a random sequence (i.e., outlier) than part of T.
- The **HON-based detector** [3,21,30] uses a network representation that accurately and flexibly embeds variable orders of dependencies and is proposed to overcome the information loss of conventional network representations. HON-based anomaly detection methods use graph distance metrics to surveil the network change with and without a test sequence. If the distance is larger than a given threshold, then it is identified as an anomaly because it appears to have unique data patterns.

4.2. Synthetic EHR dataset

4.2.1. Dataset description

The primary advantage of using synthetic data is that the ground truth is known, which provides the what, when, and where for anomalies in the data. Fig. 5 illustrates the process of building the synthetic EHR data. The goal is to synthesize sequences with variable orders of EHRs. At the beginning, we assume there are 15 events, which are then organized as a 3 × 5 grid and numbered from 0 to 14. Fig. 5a shows the process of building the synthetic normal data. The authors assume that there are five steps for choosing five events in each sequence, in which one of three events in each column is selected at each step with the same possibility, except for three defined constraints. **First-order constraint** (depicted by a red-dashed arrow): if the previous event is 0, then the next event must be 3 or 4 with equal transition probability. **Second-order constraint** (depicted by blue-dashed arrow): if the current event is 9, and the previous event is 6, then the event has a 50% chance of moving to event 12 and a 50% chance of moving to event 13. **Third-order constraint** (depicted by green-dashed arrow): if the previous movements are 2 → 5 → 8, then the next event will be event 11. The authors also assume that the duration of each transition is 1 min during this process. One thousand sequences are generated as normal data to simulate 5000 events.

Fig. 5b shows the five synthetic anomalous sequences with their abnormal transitions labeled with colors that correspond to the rules they violated. Sequences 1001, 1002, and 1003 did not follow the first-order, second-order, and third-order constraints, respectively. For sequences 1004 and 1005, the authors injected abnormal duration, in which the transition duration of 9 → 12, 10 → 13 is 10 min rather than the 1 min for a normal sequence.

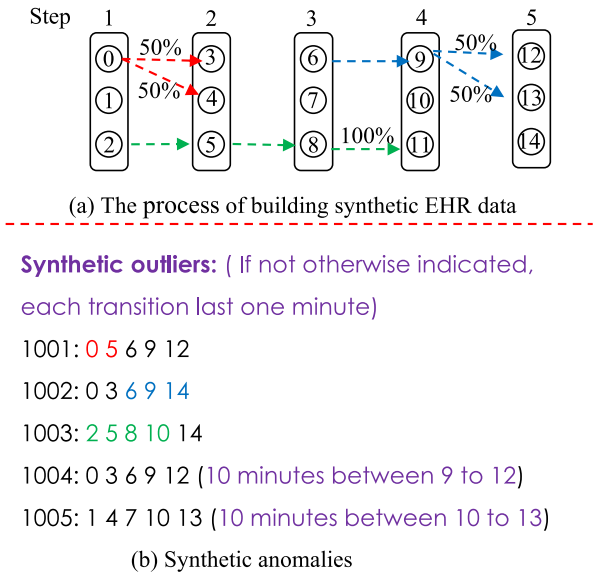


Fig. 5. Building synthetic EHR data. (a) Process of building normal data in which one of the three events is selected in each step and five events in each sequence data. Three rules are specified using red-, blue-, and green-dashed arrows. (b) Five synthetic anomalies with abnormal transitions are also labeled using their corresponding colors. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 1
Detection results for synthetic dataset: *m* means miss (i.e., not detected), and *h* means hit (i.e., detected).

ID	kNN	k-means	N-gram	PST	ADHON	AD-HOTN
1001	h	m	h	h	h	h
1002	h	h	h	h	h	h
1003	h	h	h	m	h	h
1004	m	m	m	m	m	h
1005	m	m	m	m	m	h

4.2.2. Experimental settings

In the experiment, SGT was used as the embedding method for the three point-based anomaly detection methods. All methods were implemented in Python 3.6 and tested on a machine with 16 GB of memory. For kNN, the nearest 1% neighbor points were chosen for the $n_{neighbors}$ parameter by using Euclidean distance as a distance metric. For k-means, the original data were clustered to 100 clusters. The event length of N-gram was set to 5, and the maximum depth of PST was set to 5.

4.2.3. Results

Table 1 presents the detection results for synthetic data using five baseline methods and the proposed AD-HOTN approach, where *m* means miss (i.e., not detected), and *h* means hit (i.e., detected). The first three point-based methods are applied using SGT as the embedding method. As shown in the table kNN detected the first three anomalies, whereas k-means, and PST detected two anomalies each. ADHON and N-gram both detected three anomalies. None of the baseline methods detected anomalies with abnormal duration. By comparison, AD-HOTN detected anomalies with unique transition patterns and those with abnormal durations.

CSAD-HOTN detects additional anomalous subsequences and recommends plausible transitions for each anomaly. Table 2 shows these findings in more detail with different colors highlighting the transitions. For sequence 1001, the first-order transition 0→5 is identified, and 0→4 and 0→3 are given as plausible transitions with each near a 50% transition probability. For sequence 1002, synthetic abnormal transition 9|6→14 is identified, and 9|6→12 and 9|6→13 are given as plausible transitions with 50% transition probability. As for the third-order abnormal transition, 8|5.2 →10 is also flagged correctly with 8|5.2 →11 given as the plausible transition with 1.0 probability. For the last two outliers, 10 min abnormal duration is captured for transition 9→12 and 7→10. The plausible transition duration is reported as 1 min. In summary, CSAD-HOTN accurately detected all synthetic abnormal subsequences, and the recommended transitions were correctly identified.

Table 2
Abnormal transitions and recommended transitions for synthetic EHR dataset by CSAD-HOTN.

ID	Abnormal transitions (source → target, tp, duration)	Recommend transitions (source → target, tp, duration)
1001	0 → 5, 0.0028, 0:01:00	0 → 4, 0.527, 0:01:00 0 → 3, 0.473, 0:01:00
1002	9|6 → 14, 0.009, 0:01:00	9|6 → 12, 0.5, 0:01:10 9|6 → 13 ,0.5, 0:01:00
1003	8|5.2 → 10, 0.038, 0:01:00	8|5.2 → 11, 1.0, 0:01:00
1004	9 → 12, 0.337, 0:10:00	9 → 12, 0.337, 0:01:00
1005	7 → 10, 0.302, 0:10:00	7 → 10, 0.302, 0:01:00

4.2.4. Performance comparison on larger synthetic datasets

For a more in-depth comparison of the algorithm's performance vs. the baseline methods, the authors also applied the algorithms to five synthetic datasets, each of which contained 100,000 normal sequences and 500 anomalous sequences. Specifically, for each dataset, the normal data are generated by using the same method shown in Fig. 5a. For the synthetic anomalies in each dataset, the authors generated 400 outliers that contained abnormal transitions and 100 outliers that contained abnormal duration. When generating the outlier that contains abnormal transitions, one of the three transitions rules in Fig. 5a (depicted as red-, blue-, and green-dashed arrows) was violated randomly. Then, the five datasets were tested with AD-HOTN and five baseline methods.

Performance Metrics. To evaluate the performance of the anomaly detection methods, we use three standard metrics: Precision, Recall, and F-score. These metrics are computed as follows:

$$Precision = \frac{TP}{TP + FP}, \tag{9}$$

$$Recall = \frac{TP}{TP + FN}, \tag{10}$$

$$F - score = \frac{2 * TP}{2 * TP + FP + FN}, \tag{11}$$

where TP means true positive, FP means false positive, and FN means false negative. Precision shows the percentage of true anomalies in all detected anomalies. Recall depicts the percentage of real anomalies detected. F-score is the harmonic mean of Precision and Recall.

To show the robustness and eliminate random system errors, we conducted five runs for each method in which synthetic data is generated randomly for each run. The results show that no FPs appeared for any methods in any run, which means their Precisions are 1. However, they exhibited different performance for FNs. Table 3 presents the recall values of each run for the proposed method vs. baseline methods. According to the results, the only method that significantly improves detection accuracy is the AD-HOTN model, which has the best performance in each run. The next-best method detected only 80% of the outliers; specifically, it detected those with abnormal transitions. Interestingly, the performance of the kNN-based approach and of AD-HON is almost the same with 79.6% average recall value and a 0.8% STD. Among the three point-based approaches, kNN performs the best and is the most stable. For the three Markov-based baseline methods, the average recall of ADHON is 79.6%, which means it performs better

Table 3
Test recall for each method on synthetic data.

Run	kNN	k-means	N-gram	PST	ADHON	AD-HOTN
1	78%	48%	76%	50%	80%	**100%**
2	80%	44%	80%	54%	80%	**100%**
3	80%	50%	80%	48%	78%	**98%**
4	80%	66%	80%	58%	80%	**100%**
5	80%	74%	78%	58%	80%	**100%**
Average	79.6%	56.4%	78.8%	53.6%	79.6%	**99.6%**
STD	0.8%	11.5%	1.6%	4.1%	0.8%	**0.8%**

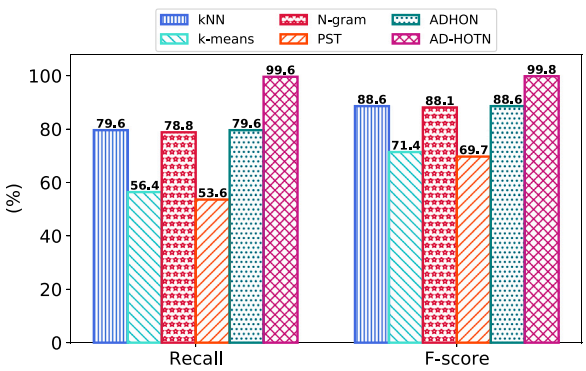


Fig. 6. Performance comparison between AD-HOTN and baselines for large synthetic datasets. The Precision for each method is 1.

than N-gram and PST. AD-HOTN detected abnormal transitions and abnormal durations. Fig. 6 compares the performance of AD-HOTN with five baselines. Each method's precision is not presented because they are all 1. We note that the F-score of AD-HOTN is the highest because it detects anomalies that only contain abnormal durations. k-NN, N-gram, and AD-HON have similar performance for anomalies with normal transition durations.

4.3. Real-world EHR dataset

4.3.1. Experimental setup and dataset

The EHR data collected in this paper were obtained from the VA's Corporate Data Warehouse (CDW). In the absence of well-defined event logs in the CDW, the state transition diagram of the OASIS Human Task standard [39] was adopted to map raw EHR log data collected in a healthcare setting. The raw data were mapped to the OASIS standard by using the rules developed in consultation with the domain experts at the VA [40,41]. The data were mapped into eight unique events: (1) `Created`, (2) `Ready`, (3) `Reserved`, (4) `InProgress`, (5) `Completed`, (6) `Failed`, (7) `Exited`, and (8) `END`. The dataset is a collection of records prepared for a cohort of approximately 808,000 patients diagnosed with ischemic heart disease between January 1, 2017 and December 31, 2019. The authors extracted 200,000 sequences in which the number of events per sequence was between 3 and 18 to test the proposed methods.

4.3.2. Data preprocessing and parameter settings

The frequency of sequences can be as high as 20,000 and as low as 1. First, to drop random noise, we remove the data that has a frequency of less than 30 from the dataset. After preprocessing, 120,000 normal sequences are extracted. We then inject different ratios (e.g., 5%, 10%, 15%, 20%) of real-world anomalies into the normal sequence for testing. In the injected anomalies, 90% of them contain real-world abnormal workflows only, and 10% of them contain abnormal duration in normal workflows where the duration of a transition is substantially larger or smaller than those of normal sequences. We run a script to automatically select real-world abnormal workflows from raw data and generate the abnormal duration by randomly changing a duration for a normal sequence. In the experiment, we use the same parameters for synthetic data.

4.3.3. Results and discussions

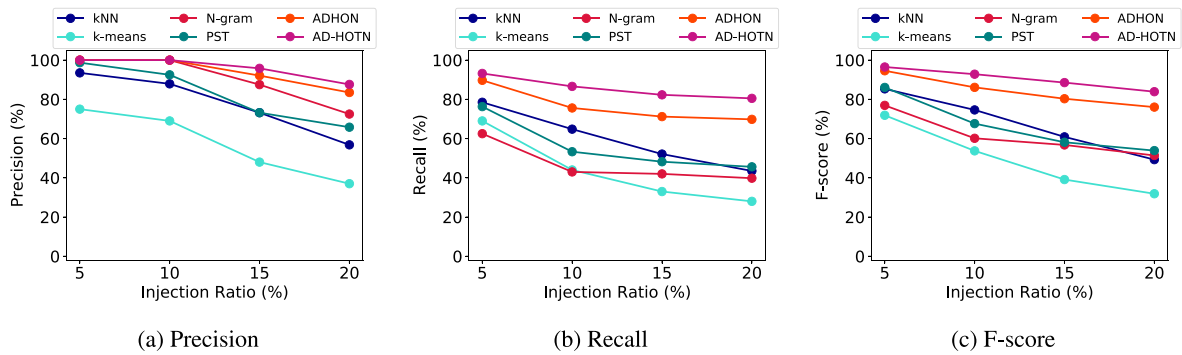
We compare the anomaly detection performance of AD-HOTN with the five baseline approaches on the real-world EHR dataset with four different anomaly injection ratios. The experimental results are shown in Fig. 7. AD-HOTN performs much better than other approaches. With the increasing injection ratio of anomalies, the performance of the related approaches has declined in different degrees. However, AD-HOTN still maintains a high accuracy even under a high injection ratio. For example, AD-HOTN can still achieve an F-score of 0.83 when the injection ratio is 20% (i.e., when 20% of the overall sequences are intentional injected anomalies). It confirms that our approach is robust enough for real-world scenarios. The reason is that AD-HOTN can capture higher-order dependencies and their durations through HOTN representations, so it can identify hard-to-detect anomalies. Traditional approaches cannot support flexible higher-order growth when needed and are thus unable to achieve satisfactory results.

Second, we conducted three case studies to understand the duration detection using AD-HOTN on real-world data with a 10% injection ratio. The details are shown in Fig. 8. In each case study, the transitions have the same start and end events but different dependencies. By considering the previous start events, AD-HOTN can identify those anomalies that were normal with different dependencies (some of them are labeled using red circles). Fig. 8(a) shows four transitions that were generated by AD-HOTN from event `InProgress` to event `Completed`, but their dependencies are not the same. The durations of the four transitions have different distributions. From top to bottom, the four dots labeled with red circles are 22.3, 53.7, 23.1, and 43.2. The dots can be normal if the higher-order dependencies are not considered because they will hide in the middle of the data after stacking all the data points shown in Fig. 8(a). Specifically, for the circled durations in the first (blue) and third (green), they can be normal if their transition dependencies are `Ready` (orange). Similarly, for the circled durations in the second (orange) and fourth transitions (red), they can be normal if their transition dependencies are `Ready.Reserved` (green). The same scenarios are also found in Figs. 8(b) and 8(c). With different order dependencies built by AD-HOTN as needed, the salient information about different distributions is captured that helps identify anomalies that might be ignored by traditional approaches.

4.4. Performance scalability

The authors further demonstrated the scalability advantage of Spark-based AD-HOTN by using different-sized datasets from real-word EHR data extracted from the CDW. For comparison, AD-HOTN with a different number of executors and ADHON were deployed on identical hardware (Intel i5 quad-core CPU at 2.3 GHz, 16 GB of RAM, and a solid-state storage drive).

Fig. 9 shows how these algorithms scale across a range of sequences for a different number of executors. For AD-HOTN, more nodes means less run time and better performance, especially as the data size increases above 60,000 cases. For comparison, running ADHON on a single node for the same dataset is also presented. Running Spark-based AD-HOTN on any data size is considerably faster than ADHON with more than one executor. The run-time speedup between AD-HOTN with eight nodes and ADHON is also presented in Fig. 9, which demonstrates the scalability of the Spark-based AD-HOTN algorithm.



(a) Precision (b) Recall (c) F-score

Fig. 7. Comparison of anomaly detection accuracy between AD-HOTN and the baselines with four different anomaly injection ratios and a real-world dataset.

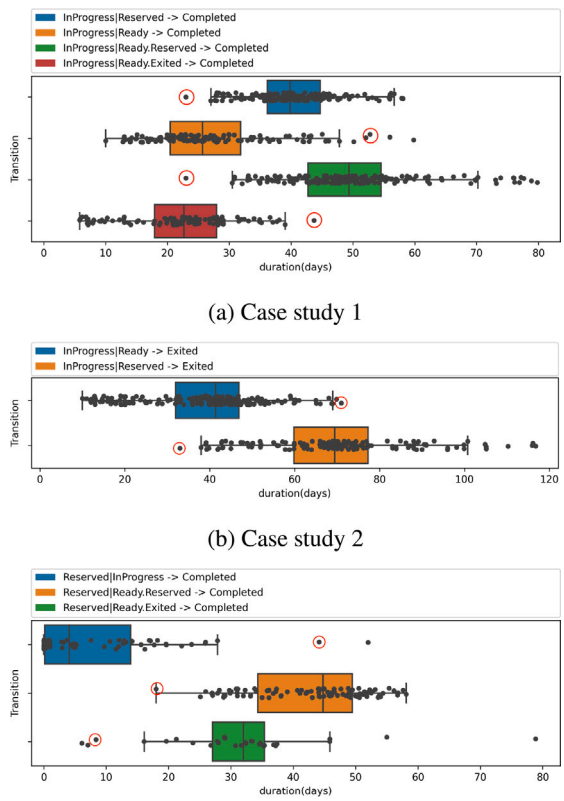


Fig. 8. Three case studies using AD-HOTN on real-world data with a 10% injection ratio. In each case study, the transitions have the same start and end events but different dependencies. By considering the previous start events, AD-HOTN can identify those anomalies that were normal with different dependencies (some of them are labeled using red circles). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

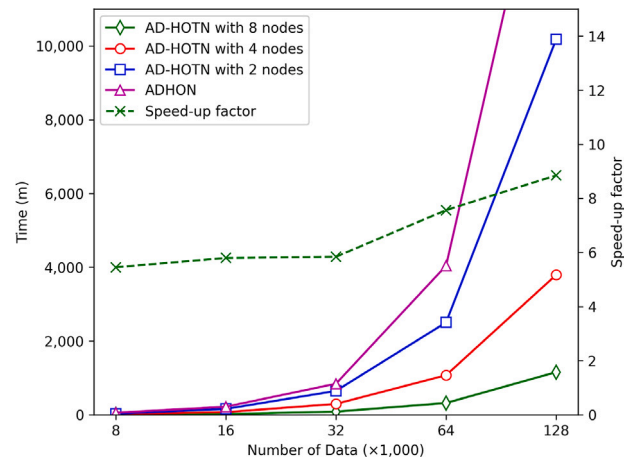


Fig. 9. Performance comparison across different data sizes and approaches.

4.5. Importance of anomaly detection for HIT systems

The widespread adoption of information technology in health care, which was mandated by the US Health Information Technology for Economic and Clinical Health Act of 2009, has made creating an ecosystem of safer health care one of its goals. Because of this regulation, the use of HIT in US health care facilities has increased from 12.2% in 2009 to around 91% by 2017 [42]. This rapid HIT adoption has facilitated efforts to increase the quality and efficiency of health care services and decrease health care overhead while simultaneously generating massive amounts of EHRs [43]. Although HIT presents many new opportunities for improving patient care and safety, it can also create new challenges and vulnerabilities [44]. Recent studies that investigated the vulnerabilities in HIT [42–44] describe how data transmitted through the HIT systems could alter clinical decisions, interrupt the sequence of care, and/or alter care delivery processes. In a 10-year study of HIT related incidents, Martin et al. [45] found that 82% of incidents caused no harm to patients, 13% caused low harm, 4% caused moderate harm, 1% caused severe harm and less than 1% caused death. In a survey of 229 incidents (from 249 reports) from the U.S. Food and Drug Administration Manufacturer and User Facility Device Experience database, Marcilly et al. [46] determined that 45.5% of the incidents resulted in no patient injury. However, 12.7% (29 incidents) resulted in harm to the patient including 3 deaths (1.3%). The percentages of severe harm and death appear low but given the high number of HIT related incidents (e.g., 2627 in England and Wales over the study's 10-year period [45]), the amount of harm is significant.

Anomaly detection techniques for healthcare sequential data can mitigate any potential patient safety hazards by detecting irregular patterns in the HIT data. This will help support improved decisions by clinicians, informaticists, IT specialists, engineers (e.g., software, systems), data scientists, and social scientists. Improvements in HIT hazard anomaly detection and mitigation have the potential to alleviate enormous suffering. This paper presents new event sequence (and subsequence) anomaly detection algorithms that account for variable higher-order dependencies in the HIT data.

In this study, we are interested in revealing infrequent patterns in the EHR data that can be potential hazards. In particular, we are interested in clinical orders that have incomplete outcomes such as the Error and Failed states. For example, proactively predicting the final state of clinical orders while they are still ongoing is one use case of interest. There are hundreds of thousands of orders per clinical

domain (e.g., radiology, laboratory, etc.) generated daily in the VA EHR system. The objective is to identify possible Error or Failed clinical orders beforehand so that appropriate interventions can probably be performed to avoid a potential unsuccessful outcome. For instance, assume that an EHR system software glitch prevented some clinical orders from going through their usual route. This may create different event sequences that are rare, and they have never been observed in the HIT system. These sequences will be detected using the proposed approach and tagged for further analysis.

5. Related work

Anomaly detection methods for event sequences have been studied extensively over the last decade. The following two sections briefly review the related methods.

5.1. Point-based anomaly detection algorithm

Many studies have been conducted on event-sequence anomaly detection using point-based methods [9–12]. These methods calculate the similarity between a test sequence and training sequences and use the dissimilarity scores as anomaly scores. Chandola et al. [10] proposed a kNN-based method that calculates the dissimilarity to its kth nearest neighbor. K-means, N-gram, and other clustering-based methods have been discussed in the literature [34–36]. However, with those methods, the effectiveness of the detection process can be affected by the similarity metrics. On the other hand, many proposed feature-embedding methods can largely improve the performance of the traditional methods [13–16]. Ranjan et al. [14] proposed an effective embedding method, SGT, which can extract a varying amount of short- to long-term dependencies with high accuracy when used with traditional point-based approaches.

5.2. Markov-based anomaly detection algorithm

Markov-based methods are popular sequence anomaly detection approaches that use the short-term property of sequences by building a probabilistic model from the conditional probability for events. Boldt et al. [47] applied Markov models on streaming sequence data by considering different temporal resolutions, but they used fixed-order Markov models. The fixed-Markovian approach has limitations because the methods use a fixed history of a sequence for building the probabilistic model, and the methods always require a huge amount of storage [17,18]. Variable Markov-based methods [3,20,21] are proposed to address these limitations. The PST was proposed by Sun et al. [19] for representing a variable-length history of a sequence as a compact tree representation. The study by Niu et al. [21] took dynamic anomalies into consideration using variable-order network representations. Xu et al. [20] proposed a sequence anomaly detection algorithm for detecting new events of data by building an efficient variable Markov-based network representation that generated higher-order dependencies on demand. Nevertheless, these methods focus only on the events of the sequences and ignore the additional information (e.g., event duration) associated with the events.

Jin et al. [48] demonstrated the edge-labeled graph readability query problem in which edges are associated with labels to denote different types of relationships between vertices. This method also demonstrates the importance of label/salient information in graph problems. Xie et al. [29] discussed the advantage of using tensor graph models for traffic-monitoring data. Their model adds time between traffic data as a tensor to their graph model, which can provide an accurate recovery model for the traffic data. However, the model does not consider the higher-order dependencies in the data.

6. Conclusion

This paper proposes higher-order tensor network-based event sequence (AD-HOTN) and continuous subsequence (CSAD-HOTN) anomaly detection algorithms. The algorithms handle high-dimensional event sequence data by embedding event transitions and associated information about the events into variable orders of network representation. Specifically, the authors define anomalies as sequences that cause a large network change when removed from the test dataset; the network change is then quantified using graph distance metrics. The algorithms were tested using synthetic and real-world EHR data, and the performance of AD-HOTN was compared with that of five baseline methods. The AD-HOTN algorithm outperformed the baseline methods, and the CSAD-HOTN algorithm correctly identified the plausible transitions for the detected anomalous subsequences in the abnormal sequences.

The work reported here has potential applications well beyond electronic health records. In the life sciences alone, anomaly detection can help highlight crucial risks and identify potential problems in patient care. As an illustration, an anomalous sequence of clinical actions may pinpoint a worrisome aberration from standard treatment regimens. Numerous examples have been presented in the literature. Anomalies in transcriptomic data may elucidate previously-unknown phenomena that warrant further study [49]. Anomalous metabolomic signatures can aid in the diagnosis of Central Nervous System diseases [50]. Anomalies in medication adherence patterns might also indicate behavioral instability [51].

Methods such as AD-HOTN and CSAD-HOTN that rely on network change to detect anomalies should perform well in these and related applications. Implications for advances in human health are manifest. Future work will involve conducting experiments on the data with more salient information between graph edges and performing experiments on larger real-world datasets.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] V. Chandola, A. Banerjee, V. Kumar, Anomaly detection for discrete sequences: A survey, IEEE Trans. Knowl. Data Eng. 24 (5) (2010) 823–839.
[2] R. Athilakshmi, S.G. Jacob, R. Rajavel, Protein sequence based anomaly detection for neuro-degenerative disorders through deep learning techniques, in: Advances in Big Data and Cloud Computing, Springer, 2019, pp. 547–554.
[3] H. Niu, O. Omitaomu, Q. Cao, O. Ozmen, H. Klasky, M.M. Olama, L. Pullum, T. Kuruganti, M. Ward, A. Laurio, et al., Anomaly detection in sequential health care data using higher-order network representation, Tech. rep., Oak Ridge National Laboratory, Oak Ridge, TN, United States, 2020.
[4] H. Estiri, J.G. Klann, S.N. Murphy, A clustering approach for detecting implausible observation values in electronic health records data, BMC Med. Inform. Decis. Making 19 (1) (2019) 1–16.
[5] B.J. Radford, B.D. Richardson, S.E. Davis, Sequence aggregation rules for anomaly detection in computer network traffic, 2018, arXiv preprint arXiv:1805.03735.
[6] B.J. Radford, L.M. Apolonio, A.J. Trias, J.A. Simpson, Network traffic anomaly detection using recurrent neural networks, 2018, arXiv preprint arXiv:1803.10769.
[7] N. Malini, M. Pushpa, Analysis on credit card fraud identification techniques based on KNN and outlier detection, in: 2017 Third International Conference on Advances in Electrical, Electronics, Information, Communication and Bio-Informatics (AEEICB), IEEE, 2017, pp. 255–258.
[8] V. Saligrama, J. Konrad, P.-M. Jodoin, Video anomaly identification, IEEE Signal Process. Mag. 27 (5) (2010) 18–33.
[9] S. Ramaswamy, R. Rastogi, K. Shim, Efficient algorithms for mining outliers from large data sets, in: Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, 2000, pp. 427–438.
[10] V. Chandola, V. Mithal, V. Kumar, Comparative evaluation of anomaly detection techniques for sequence data, in: 2008 Eighth IEEE International Conference on Data Mining, IEEE, 2008, pp. 743–748.

[11] G. Münz, S. Li, G. Carle, Traffic anomaly detection using k-means clustering, in: GI/ITG Workshop MMBnet, 2007, pp. 13–14.
[12] C. Aytekin, X. Ni, F. Cricri, E. Aksu, Clustering and unsupervised anomaly detection with l 2 normalized deep auto-encoder representations, in: 2018 International Joint Conference on Neural Networks (IJCNN), IEEE, 2018, pp. 1–6.
[13] K.W. Church, Word2Vec, Natural Language Eng. 23 (1) (2017) 155–162.
[14] C. Ranjan, S. Ebrahimi, K. Paynabar, Sequence graph transform (SGT): A feature embedding function for sequence data mining, 2016, arXiv preprint arXiv:1608.03533.
[15] A. Graves, Generating sequences with recurrent neural networks, 2013, arXiv preprint arXiv:1308.0850.
[16] Z. Wang, Z. Chen, J. Ni, H. Liu, H. Chen, J. Tang, Multi-scale one-class recurrent neural networks for discrete event sequence anomaly detection, 2020, arXiv preprint arXiv:2008.13361.
[17] C.C. Michael, A. Ghosh, Two state-based approaches to program-based anomaly detection, in: Proceedings 16th Annual Computer Security Applications Conference (ACSAC'00), IEEE, 2000, pp. 21–30.
[18] C. Marceau, Characterizing the behavior of a program using multiple-length N-grams, in: Proceedings of the 2000 Workshop on New Security Paradigms, 2001, pp. 101–110.
[19] P. Sun, S. Chawla, B. Arunasalam, Mining for outliers in sequential databases, in: Proceedings of the 2006 SIAM International Conference on Data Mining, SIAM, 2006, pp. 94–105.
[20] J. Xu, M. Saebi, B. Ribeiro, L.M. Kaplan, N.V. Chawla, Detecting anomalies in sequential data with higher-order networks, 2017, arXiv preprint arXiv:1712.09658.
[21] H. Niu, O.A. Omitaomu, Q.C. Cao, M. Olama, O. Ozmen, H. Klasky, L. Pullum, A.T. Malviya, T. Kuruganti, J. Scott, et al., Adaptive anomaly detection for dynamic clinical event sequences, in: 2020 IEEE International Conference on Big Data, IEEE, 2020, pp. 4919–4928.
[22] S.-B. Cho, H.-J. Park, Efficient anomaly detection by modeling privilege flows using hidden Markov model, Comput. Secur. 22 (1) (2003) 45–55.
[23] S.S. Joshi, V.V. Phoha, Investigating hidden Markov models capabilities in anomaly detection, in: Proceedings of the 43rd Annual Southeast Regional Conference, Volume 1, 2005, pp. 98–103.
[24] X. Tan, H. Xi, Hidden semi-Markov model for anomaly detection, Appl. Math. Comput. 205 (2) (2008) 562–567.
[25] M. Du, F. Li, G. Zheng, V. Srikumar, Deeplog: Anomaly detection and diagnosis from system logs through deep learning, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, 2017, pp. 1285–1298.
[26] A. Brown, A. Tuor, B. Hutchinson, N. Nichols, Recurrent neural network attention mechanisms for interpretable system log anomaly detection, in: Proceedings of the First Workshop on Machine Learning for Computing Systems, 2018, pp. 1–8.
[27] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, T. Soderstrom, Detecting spacecraft anomalies using lstms and nonparametric dynamic thresholding, in: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2018, pp. 387–395.
[28] D. Gusfield, Algorithms on stings, trees, and sequences: Computer science and computational biology, ACM Sigact News 28 (4) (1997) 41–60.
[29] K. Xie, X. Li, X. Wang, G. Xie, J. Wen, D. Zhang, Graph based tensor recovery for accurate internet anomaly detection, in: IEEE INFOCOM 2018-IEEE Conference on Computer Communications, IEEE, 2018, pp. 1502–1510.
[30] J. Xu, T.L. Wickramarathne, N.V. Chawla, Representing higher-order dependencies in networks, Sci. Adv. 2 (5) (2016) e1600028.
[31] D. Ron, Y. Singer, N. Tishby, The power of amnesia: Learning probabilistic automata with variable memory length, Mach. Learn. 25 (2) (1996) 117–149.
[32] P. Shoubridge, M. Kraetzl, W. Wallis, H. Bunke, Detection of abnormal change in a time series of graphs, J. Interconnect. Netw. 3 (01n02) (2002) 85–101.
[33] M. Zaharia, R.S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M.J. Franklin, et al., Apache spark: a unified engine for big data processing, Commun. ACM 59 (11) (2016) 56–65.
[34] W. Zhong, G. Altun, R. Harrison, P.C. Tai, Y. Pan, Improved K-means clustering algorithm for exploring local protein sequence motifs representing common structural property, IEEE Trans. Nanobiosci. 4 (3) (2005) 255–265.
[35] M. Çelik, F. Dadaşer-Çelik, A.Ş. Dokuz, Anomaly detection in temperature data using dbscan algorithm, in: 2011 International Symposium on Innovations in Intelligent Systems and Applications, IEEE, 2011, pp. 91–95.
[36] S. Budalakoti, A.N. Srivastava, M.E. Otey, Anomaly detection and diagnosis algorithms for discrete symbol sequences with applications to airline safety, IEEE Trans. Syst. Man Cybern. Part C 39 (1) (2008) 101–113.
[37] C. Manning, H. Schutze, Foundations of Statistical Natural Language Processing, MIT Press, 1999.
[38] N. Hubballi, S. Biswas, S. Nandi, Sequencegram: n-gram modeling of system calls for program based anomaly detection, in: 2011 Third International Conference on Communication Systems and Networks (COMSNETS 2011), IEEE, 2011, pp. 1–10.
[39] Oasis, web services human task (ws-human task) specification version 1.1 – committee specification draft 12/ public review draft 05, 2012.
[40] H.B. Klasky, O. Ozmen, Process Mining in Healthcare-A Case Study for the Corporate Data Warehouse of the Veterans Affairs Office, Tech. rep., Oak Ridge National Laboratory, Oak Ridge, TN (United States), 2019.
[41] O. Ozmen, H.B. Klasky, O.A. Omitaomu, M. Olama, T. Kuruganti, M. Ward, J.M. Scott, A. Laurio, F. Drews, J.R. Nebeker, Feature engineering and process mining to enable hazard detection in health information technology, in: AMIA Summits on Translational Science Proceedings 2020, 2020, p. 469.
[42] T. Heart, O. Ben-Assuli, I. Shabtai, A review of PHR, EMR and EHR integration: A more personalized healthcare and public health policy, Health Policy and Technol. 6 (1) (2017) 20–25.
[43] M.O. Kim, E. Coiera, F. Magrabi, Problems with health information technology and their effects on care delivery and patient outcomes: a systematic review, J. Am. Med. Inform. Assoc. 24 (2) (2017) 246–250.
[44] S.K. Brenner, R. Kaushal, Z. Grinspan, C. Joyce, I. Kim, R.J. Allard, D. Delgado, E.L. Abramson, Effects of health information technology on patient outcomes: a systematic review, J. Amer. Med. Inform. Assoc. 23 (5) (2016) 1016–1036.
[45] G. Martin, S. Ghafur, I. Cingolani, J. Symons, D. King, S. Arora, A. Darzi, The effects and preventability of 2627 patient safety incidents related to health information technology failures: a retrospective analysis of 10 years of incident reporting in England and Wales, Lancet Digital Health 1 (3) (2019) e127–e135.
[46] R. Marcilly, J. Schiro, M.C. Beuscart-Zéphir, F. Magrabi, Building usability knowledge for health information technology: a usability-oriented analysis of incident reports, Appl. Clin. Inform. 10 (03) (2019) 395–408.
[47] M. Boldt, A. Borg, S. Ickin, J. Gustafsson, Anomaly detection of event sequences using multiple temporal resolutions and Markov chains, Knowl. Inf. Syst. 62 (2) (2020) 669–686.
[48] R. Jin, H. Hong, H. Wang, N. Ruan, Y. Xiang, Computing label-constraint reachability in graph databases, in: Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data, 2010, pp. 123–134.
[49] C. Ma, C. Kingsford, Detecting, categorizing, and correcting coverage anomalies of RNA-seq quantification, Cell Syst. 9 (6) (2019) 589–599.
[50] J. Troisi, A. Landolfi, L. Sarno, S. Richards, S. Symes, D. Adair, C. Ciccone, G. Scala, P. Martinelli, M. Guida, A metabolomics-based approach for non-invasive screening of fetal central nervous system anomalies, Metabolomics 14 (6) (2018) 1–10.
[51] J. Knights, Z. Heidary, J.M. Cochran, et al., Detection of behavioral anomalies in medication adherence patterns among patients with serious mental illness engaged with a digital medicine system, JMIR Mental Health 7 (9) (2020) e21378.